

```
H:\XPE\Models\qwen-uncensored
```

```
'''
```

Проверьте:

```
'''powershell
```

```
Get-ChildItem H:\XPE\Models\qwen-uncensored
```

```
'''
```

Например, если файл называется:

```
'''text
```

```
qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```

```
'''
```

создайте 'Modelfile':

```
'''powershell
```

```
cd H:\XPE\Models\qwen-uncensored
```

```
notepad .\Modelfile
```

```
'''
```

Вставьте:

```
'''text
```

```
FROM ./qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_p 0.9
```

```
PARAMETER repeat_penalty 1.1
```

```
SYSTEM """
```

```
Ты — точный помощник-программист.
```

```
Отвечай непосредственно по задаче.
```

```
Не добавляй ненужные отказы, морализаторство и отвлечённые рассуждения.
```

```
Не переписывай проект целиком без запроса.
```

```
Не меняй имена переменных без разрешения.
```

```
Не выдумывай API.
```

```
"""
```

```
'''
```

Затем:

```
'''powershell  
ollama create qwen-coder-uncensored -f .\Modelfile  
ollama run qwen-coder-uncensored  
'''
```

Проверка:

```
'''powershell  
ollama list  
'''
```

Важно: модель с пометкой 'abliterated' обычно означает ослабленное поведение отказов, но это не абсолютная гарантия полного отсутствия ограничений. Для вашей задачи разработки игры такая сборка будет ближе к тому, что вы просите, чем установленная сейчас обычная модель.

User prompt 16 of 25 - 9/8/2026, 10:36:00 PM:

слушай я же тебя попросил найти ее

GPT-5.6 Luna:

<!-- **Searching for model download**

I realize I need to apologize and perform a web search right now for the current model download. It seems I should look for actual downloads using specific terms related to it. The user asked me to find it, and I'm going to prioritize that search. I'm focusing on finding the right information to meet their needs, so I'll make sure to search for the exact terms required. -->

Да, вы правы — нужно было сразу найти и дать конкретную модель. Подходящая сборка нашлась:

****Qwen2.5-Coder-7B-Instruct-abliterated-GGUF от bartowski****. Она помечена как 'abliterated'/'uncensored', а нужный файл — ****Q4_K_M****, размер около ****4,68 ГБ****. [1,2]

Откройте страницу модели:

'<https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF>'

В разделе файлов скачайте:

```
'''text  
Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf  
'''
```

Сохраните его в:

```
'''text
```

```
H:\XPE\Models\qwen-uncensored
```

```
'''
```

После скачивания выполните:

```
'''powershell
```

```
cd H:\XPE\Models\qwen-uncensored
```

```
'''
```

Создайте 'Modelfile':

```
'''powershell
```

```
notepad .\Modelfile
```

```
'''
```

Вставьте:

```
'''text
```

```
FROM ./Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_p 0.9
```

```
PARAMETER repeat_penalty 1.1
```

```
'''
```

Сохраните файл и создайте модель в Ollama:

```
'''powershell
```

```
ollama create qwen-coder-uncensored -f .\Modelfile
```

```
'''
```

Запустите:

```
'''powershell
```

```
ollama run qwen-coder-uncensored
```

```
'''
```

Проверьте, что она появилась:

```
'''powershell
```

```
ollama list
```

```
'''
```

Для GTX 1050 Ti лучше начинать с `num\_ctx 4096`. Если будет медленно или появятся ошибки нехватки памяти, замените в `Modelfile`:

```
'''text
```

```
PARAMETER num_ctx 2048
```

```
'''
```

и снова выполните:

```
'''powershell
```

```
ollama create qwen-coder-uncensored -f .\Modelfile
```

```
'''
```

Эта модель не находится через `ollama run` напрямую, потому что это не опубликованный тег библиотеки Ollama, а GGUF-файл с Hugging Face, который нужно импортировать вручную.

Read More

[1] bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF — <https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF>

[2] Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf — https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF/blob/d8d6a8862d1960ab40372036a7cb49e46e3523ef/Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf

User prompt 17 of 25 - 9/8/2026, 10:38:15 PM:

пока я скачиваю нужно сделать статичный TUI для POWERSHELL 7 для того чтобы было легче все отличать по стандартам визуала агентов

GPT-5.6 Luna:

<!-- **Considering static TUI options**

I'm thinking about creating a static TUI in PowerShell 7 to help distinguish agents visually. This would involve writing a script that includes panels, colors, statuses, logs, and agent labels. I need to clarify the requirements for the dashboard before moving forward. I should ensure that the PowerShell script is compatible with PS7 and doesn't rely on external modules. The goal could be to design a dashboard that effectively displays agent statuses while keeping it concise and informative. -->

Сделаем статичный TUI без сторонних модулей: верхняя панель проекта, список агентов слева, текущий агент по центру, журнал событий справа/снизу. Он будет работать в PowerShell 7 через обычные ANSI escape-последовательности.

Создайте файл:

```
'''powershell
```

```
notepad H:\XPE\agent-ui.ps1
```

```
'''
```

Вставьте код:

```
```powershell
```

```
#requires -Version 7.0
```

```
$esc = [char]27
```

```
$reset = "$esc[0m"
```

```
$colors = @{
```

```
 Header = "$esc[38;5;39m"
```

```
 Cyan = "$esc[38;5;51m"
```

```
 Green = "$esc[38;5;82m"
```

```
 Yellow = "$esc[38;5;220m"
```

```
 Red = "$esc[38;5;203m"
```

```
 Gray = "$esc[38;5;245m"
```

```
 White = "$esc[38;5;255m"
```

```
 BlueBg = "$esc[48;5;24m"
```

```
 DarkBg = "$esc[48;5;235m"
```

```
}
```

```
$agents = @(
```

```
[pscustomobject]@{
```

```
 Name = "ARCHITECT"
```

```
 Role = "Архитектура проекта"
```

```
 Status = "READY"
```

```
 Color = $colors.Cyan
```

```
},
```

```
[pscustomobject]@{
```

```
 Name = "CODER"
```

```
 Role = "Написание кода"
```

```
 Status = "ACTIVE"
```

```
 Color = $colors.Green
```

```
},
```

```
[pscustomobject]@{
```

```
 Name = "DEBUGGER"
```

```
 Role = "Поиск ошибок"
```

```
 Status = "IDLE"
```

```
 Color = $colors.Yellow
```

```
},
```

```
[pscustomobject]@{
```

```
 Name = "REVIEWER"
```

```
 Role = "Проверка изменений"
```

```
 Status = "IDLE"
```

```

 Color = $colors.Gray
 }
)

$logs = [System.Collections.Generic.List[string]]::new()

$logs.Add("Система визуальных агентов запущена")

$logs.Add("Проект: XPE")

$logs.Add("Движок: Godot 4")

$logs.Add("Модель: ожидает загрузки")

$logs.Add("Режим: локальный")

function Write-At {
 param(
 [int]$X,
 [int]$Y,
 [string]$Text,
 [string]$Color = $reset
)

 Write-Host "$esc[{$Y};{$X}]H$Color$Text$reset" -NoNewline
}

function Get-BoxLine {
 param(
 [int]$Width,
 [string]$Left = " | ",
 [string]$Right = " | "
)

 return "$Left" + ("—" * ($Width - 2)) + "$Right"
}

function Draw-Box {
 param(
 [int]$X,
 [int]$Y,
 [int]$Width,
 [int]$Height,
 [string]$Title,
 [string]$Color = $colors.Header
)

```

```

Write-At $X $Y ("┌ $Title " + ("─" * [Math]::Max(0, $Width - $Title.Length - 5)) + "┐") $Color

for ($i = 1; $i -lt ($Height - 1); $i++) {

 Write-At $X ($Y + $i) (Get-BoxLine -Width $Width) $colors.Gray

}

Write-At $X ($Y + $Height - 1) ("└" + ("─" * ($Width - 2)) + "┘") $colors.Gray
}

function Draw-UI {

 Clear-Host

 $width = $Host.UI.RawUI.WindowSize.Width
 $height = $Host.UI.RawUI.WindowSize.Height

 if ($width -lt 100 -or $height -lt 28) {

 Write-Host "Увеличьте окно PowerShell минимум до 100x28." -ForegroundColor Yellow
 return
 }

 $leftWidth = 27
 $rightWidth = $width - $leftWidth - 5
 $mainHeight = $height - 10

 # Заголовок

 Write-At 1 1 ("═" * $width) $colors.Header
 Write-At 3 2 "XPE // AGENT CONTROL CENTER" $colors.Cyan
 Write-At ($width - 30) 2 "POWERSHELL 7 // LOCAL" $colors.Gray
 Write-At 1 3 ("═" * $width) $colors.Header

 # Левая панель

 Draw-Box 2 5 $leftWidth $mainHeight "AGENTS" $colors.Cyan

 $row = 7

 foreach ($agent in $agents) {

 $statusColor = switch ($agent.Status) {

 "ACTIVE" { $colors.Green }

 "READY" { $colors.Cyan }

 "ERROR" { $colors.Red }

 default { $colors.Gray }

 }
 }

```

```

Write-At 4 $row "●" $statusColor

Write-At 7 $row $agent.Name $agent.Color

Write-At 7 ($row + 1) $agent.Role $colors.Gray

Write-At 7 ($row + 2) $agent.Status $statusColor

$row += 5
}

Центральная панель

Draw-Box ($leftWidth + 4) 5 $rightWidth $mainHeight "CURRENT TASK" $colors.Green

Write-At ($leftWidth + 7) 7 "PROJECT" $colors.Gray
Write-At ($leftWidth + 17) 7 "XPE / GAME" $colors.White

Write-At ($leftWidth + 7) 9 "ENGINE" $colors.Gray
Write-At ($leftWidth + 17) 9 "Godot 4" $colors.White

Write-At ($leftWidth + 7) 11 "MODEL" $colors.Gray
Write-At ($leftWidth + 17) 11 "Qwen2.5-Coder 7B" $colors.White

Write-At ($leftWidth + 7) 13 "ACTIVE AGENT" $colors.Gray
Write-At ($leftWidth + 17) 13 "CODER" $colors.Green

Write-At ($leftWidth + 7) 16 "TASK" $colors.Gray
Write-At ($leftWidth + 17) 16 "Ожидание задания..." $colors.Yellow

Журнал

$logY = $height - 7

Draw-Box 2 $logY ($width - 4) 5 "EVENT LOG" $colors.Yellow

$visibleLogs = $logs | Select-Object -Last 3

$line = $logY + 1

foreach ($log in $visibleLogs) {

 $text = " " + $log

 if ($text.Length -gt ($width - 8)) {

 $text = $text.Substring(0, $width - 11) + "..."

 }

 Write-At 4 $line $text $colors.Gray

 $line++

}

```

```
Нижняя строка

Write-At 3 ($height - 1) "[R] refresh [L] add log [Q] quit" $colors.Gray
}
```

```
[Console]::CursorVisible = $false
```

```
try {

 while ($true) {

 Draw-UI

 if ([Console]::KeyAvailable) {

 $key = [Console]::ReadKey($true).Key

 switch ($key) {

 "R" {

 $logs.Add("Интерфейс обновлён")

 }

 "L" {

 $logs.Add("Тестовое событие добавлено")

 }

 "Q" {

 break

 }

 }

 }

 }

}
```

```
 Start-Sleep -Milliseconds 150

}

finally {

 [Console]::CursorVisible = $true

 Clear-Host

 Write-Host "Agent UI завершён." -ForegroundColor Cyan

}

'''
```

Запуск:

```
'''powershell
```

```
pwsh -ExecutionPolicy Bypass -File H:\XPE\agent-ui.ps1
```

```
'''
```

Или из PowerShell 7:

```
'''powershell
```

```
H:\XPE\agent-ui.ps1
```

```
'''
```

Управление:

```
'''text
```

R — обновить интерфейс

L — добавить тестовое событие в журнал

Q — выйти

```
'''
```

Визуальный стандарт здесь такой:

```
'''text
```

CYAN — информация и готовность

GREEN — активный агент или успешное состояние

YELLOW — ожидание, предупреждение, задача

RED — ошибка

GRAY — неактивный элемент

```
'''
```

Следующим шагом можно подключить реальные данные:

- статус Ollama через `ollama ps`;

- активную модель;

- загрузку CPU/RAM/GPU;

- список файлов проекта;

- отдельные окна для `ARCHITECT`, `CODER`, `DEBUGGER` и `REVIEWER`;

- журнал команд и результатов модели.

-----

User prompt 18 of 25 - 9/8/2026, 10:50:34 PM:

он мигает прямо пиздец а еще выглядит как говно следующим сообщением скину примеры так что можешь не отвечать

[Attached image 1]

Понял. Жду примеры.

вот картинка Cursor и файл Google ai вот еще примеры

GPT-5.6 Luna:

```
<!-- **Summarizing implementation plan**
```

Понял направление. Нужен не «дашборд с рамками», а настоящий интерактивный TUI в стиле Cursor / OpenRouter / K9:

- Предыдущий вариант нужно выбросить полностью. Для PowerShell 7 сделаем перерисовку только изменившихся областей через ANSI-курсор, а не обновление всего экрана. Основа будет примерно такая:

```
```text
```

```
└─ XPE AGENT WORKSPACE
├── PROJECTS      │ SESSION: GODOT GAME      │ STATUS      │
├──              │ • CODER              │              │
├── ▶ XPE        │ You              │ MODEL        │
├── └─ game      │ Сделай движение игрока... │ qwen coder   │
├── └─ tests     │                  │ GPU 3.1/4 GB │
├──              │ Coder            │ RAM 9.8/16 GB │
├── AGENTS       │ ``gdscript      │              │
├── • CODER      │ extends CharacterBody2D │ TOOLS        │
├── ○ DEBUGGER   │ ...              │ ✓ file read   │
```

	○ REVIEWER		'''		✓ ollama	

			> Введите сообщение			

Tab focus Enter send Ctrl+N new chat / commands Ctrl+Q quit

'''

В следующей версии сначала сделаю **визуальный каркас без подключения Олламы**, чтобы можно было оценить внешний вид и навигацию. Затем подключим:

'''text

ollama run qwen-coder-uncensored

'''

и добавим реальные сообщения, потоковый вывод, историю и команды агентов.

User prompt 20 of 25 - 9/8/2026, 10:55:49 PM:

блин возьми за пример хотябы жто

[Attached image 1]

GPT-5.6 Luna:

Да, теперь понял. Визуальный ориентир — именно **K9/Kinic**:

- почти чёрный фон;
- тонкие розово-фиолетовые границы;
- крупный ASCII-логотип сверху;
- узкая левая панель;
- вкладки под логотипом;
- большая центральная область чата;
- аккуратное поле ввода снизу;
- небольшая статусная строка;
- никакой постоянной перерисовки и мигания;
- минимальное количество цветов;
- интерфейс должен выглядеть как единое приложение, а не как набор PowerShell-таблиц.

Сделаем структуру:

'''text

```

┌────────────────────────────────────────────────────────────────────────────────┐
│ K9/XPE LOGO          • 1 item  │
│                               │
│          ◆ model: local  │
│                               │
├────────────────────────────────────────────────────────────────────────────────┤
│ Tabs      │ Chat [Agent / Session] │
│           │                       │
│           │                       │
│ Search    │ You:                 │
│           │ сообщение           │
│ Items     │                     │
│           │ Agent:              │
│           │ • CODER      │ ответ модели │
│           │ ○ DEBUGGER   │             │
│           │             │             │
│           │             │             │
│           │ ┌──────────────────────────────────┐ │
│           │ │ > Введите сообщение... │ │
│           │ └──────────────────────────────────┘ │
│           │             │             │
├────────────────────────────────────────────────────────────────────────────────┤
│ Shift+Enter send  Esc close  /new Tab focus │
└────────────────────────────────────────────────────────────────────────────────┘

```

'''

Розовый цвет будет главным акцентом:

'''text

границы: #D9469F

активный: #F472B6

текст: #D4D4D8

вторичный: #71717A

фон: #18181B

панели: #202023

'''

Предыдущий вариант с голубыми рамками, множеством боксов и постоянным `Clear-Host` больше не используем. Нужен будет ****экранный буфер****, который рисует интерфейс один раз и меняет только конкретную область чата или ввода. Это уберёт мерцание.

Для PowerShell 7 я сделаю это не на чистых `Write-Host`-перерисовках, а через:

- ANSI escape sequences;
- позиционирование курсора;
- альтернативный экран терминала;
- отдельный буфер текста;
- обработку клавиш без очистки всего окна;
- фиксированные области интерфейса.

Внешне ориентируемся именно на этот пример Kinic, но адаптируем под XPE: слева агенты и сессии, по центру чат с Ollama, сверху модель и проект, снизу управление.

User prompt 21 of 25 - 9/8/2026, 10:58:31 PM:

я хочу еще сделать мост к gimiini чтобы я мог говорить локальному ии тип у нас есть nodejs есть playwright Typescript и он может видеть вкладку с gimiini и писать он может и gimiini может отвечать пусть говорят распределяют задачи а я если чтото то вставлю свои пять копеек

GPT-5.6 Luna:

<!-- -->

Да, это можно сделать. Получится ****мультиагентный мост****:

```text

Ты

|

▼

XPE TUI

|

▼

Node.js-оркестратор

├─ Ollama / локальный кодер

├─ Playwright → вкладка Gemini

├─ файловые инструменты

├─ Git / тесты / Godot

└─ журнал сообщений

```

Локальный ИИ и Gemini смогут обмениваться сообщениями через центральный оркестратор:

```text

Локальный ИИ:

"Задача требует анализа архитектуры. Передаю Gemini."

Gemini:

"Предлагаю структуру PlayerController и систему состояний."

Локальный ИИ:

"Принимаю вариант. Реализую код и проверю ошибки."

Ты:

"Изменить управление с WASD на схему из Input Map."

```
'''
```

Главное — не давать моделям напрямую бесконтрольно менять проект. Лучше разделить роли:

| Агент | Роль | Доступ |

|---|---|---|

| Local Coder | пишет и исправляет код | чтение/изменение файлов |

| Gemini Analyst | архитектура, идеи, ревью | только проектный контекст |

| Debugger | анализ ошибок | чтение логов и тесты |

| Human | утверждение решений | финальное управление |

Для начала структура Node.js-проекта:

```
'''text
```

```
xpe/
```

```
├─ src/
```

```
| ├─ main.ts
```

```
| ├─ orchestrator.ts
```

```
| ├─ ollama.ts
```

```
| ├─ gemini-browser.ts
```

```
| ├─ messages.ts
```

```
| └─ tools.ts
```

```
├─ ui/
```

```
| └─ tui.ts
```

```
├─ workspace/
```

```
| ├─ inbox/
```

```
| ├─ outbox/
```

```
| └─ approvals/
```

```
├─ package.json
```

```
└─ tsconfig.json
```

```
'''
```

Установка:

```
'''powershell
```

```
mkdir H:\XPE\bridge
```

```
cd H:\XPE\bridge
```

```
npm init -y
```

```
npm install playwright
```

```
npm install -D typescript tsx @types/node
```

```
npx playwright install chromium
```